

The CZOA/CZOI Tutorial

A Guided Journey from Fragmented Systems to Unified Intelligence — and to Integrated Organizations

Welcome

This tutorial will teach you to think in CZOA — the **Constrained Zoned-Object Architecture** — and to build real systems with the **CZOI toolkit**.

It is not a reference manual. It is a journey. We start with a problem that has frustrated software architects for decades — the fragmentation of modern enterprises into incompatible silos — and we arrive at a framework that lets you build systems that are simultaneously secure, intelligent, organisation-aligned, and integrable. Along the way, you'll write code that runs. By the end, you'll be able to look at any complex organisation — a hospital, a bank, a factory, a government, a post-merger enterprise — and design a CZOI system that models it faithfully and composes it cleanly.

How to use this tutorial:

- Read it sequentially. Each chapter builds on the last.
- Type every code example. Don't just read them.
- Do the exercises. They're where the learning happens.
- When you get stuck, the [User Guide](#) has the reference details.

What you'll need:

- Python 3.9 or later
- The CZOI toolkit: `pip install czo-tutorial`
- Willingness to think abstractly — but we'll make every abstraction concrete.

Estimated time: 14–24 hours to work through carefully. Each chapter takes 45–90 minutes.

Table of Contents

- [Part I — Foundations](#)
 - [Chapter 1 — The Fragmentation Problem](#)
 - [Chapter 2 — Zones: The Recursive Foundation](#)
 - [Chapter 3 — The 10-Tuple](#)
- [Part II — Core Mechanics](#)
 - [Chapter 4 — Roles, Users, and Seniority](#)
 - [Chapter 5 — Operations and the Application Boundary](#)
 - [Chapter 6 — The Permission Calculus](#)
- [Part III — Intelligence](#)
 - [Chapter 7 — Neural Components](#)
 - [Chapter 8 — Semantic Embeddings](#)
 - [Chapter 9 — Adaptive Access Control](#)
- [Part IV — Governance](#)

- Chapter 10 — Constraints via UniLog
 - Chapter 11 — Daemons and Continuous Monitoring
 - Part V — Real Systems
 - Chapter 12 — Designing for a Domain
 - Chapter 13 — Testing and Verification
 - Chapter 14 — Deployment Patterns
 - Chapter 15 — Integrating Heterogeneous Systems
 - Part VI — Advanced
 - Chapter 16 — Cross-Zone Reasoning
 - Chapter 17 — Composition and Federation
 - Chapter 18 — The Category-Theoretic View
 - Conclusion
 - Appendices
-

Part I — Foundations

Chapter 1 — The Fragmentation Problem

Learning objectives

- Understand the three ways modern enterprise systems are fragmented.
- See the gaps between "intelligent systems", "secure systems", and "integrated systems".
- Get a preview of how CZOA closes all three gaps with one mechanism.

Three engineers, one hospital network

Meet **Maya**, **Sam**, and **Priya**. They work at the same regional health authority but rarely talk.

Maya builds the AI triage system. She's an ML engineer. Her models predict sepsis risk from vital signs with 94 % accuracy. She's published papers. Her code runs on GPUs. She thinks in tensors.

Sam builds the access-control system. He's an enterprise architect. He knows the hospital's organisational chart down to the sub-team level. He's implemented RBAC, ABAC, and SoD. His code runs on a mainframe. He thinks in permissions.

Priya is the integration architect. She arrived six months ago when the regional health authority was formed by merging three previously independent hospitals. Her job is to make the merged organization work as one — without rewriting any of the legacy systems, without breaking any of the pre-merger compliance certifications, and without forcing three different clinical cultures to adopt a single set of IT tools.

When flu season hits and the hospital network needs to redeploy nurses across all three legacy hospitals, three things happen:

- Maya's sepsis model says "these patients are critical" — but it only sees patients from the hospital where it was trained.
- Sam's access system says "these nurses cannot prescribe" — because the roles from Hospital A don't exist in Hospital B's role taxonomy.

- Priya's integration work is stalled — she's been waiting six weeks for the legacy EMR vendors to expose yet another batch of API endpoints, and each one reveals a new inconsistency with the other two systems.

Patients wait. Nurses are redeployed manually via paper forms. Three weeks later, a spreadsheet is produced.

This is the fragmentation problem. Not one fragmentation, but three, each reinforcing the others.

The three fragmentations

Fragmentation 1 — Intelligence vs. security. The theories that explain intelligent behaviour and the methodologies that build secure systems have evolved separately. Maya's model is "correct" if it predicts well. Sam's system is "correct" if no unauthorised access occurs. These are not the same thing, and optimising one often degrades the other.

Fragmentation 2 — Local autonomy vs. global governance. Every subunit of an organization wants to evolve at its own rate, use its own tools, and preserve its own institutional knowledge. Every governance function wants a single policy, a single audit, a single source of truth. These goals are not compatible in traditional architectures — you get one or the other, never both.

Fragmentation 3 — Heterogeneous systems vs. coherent operations. Real organizations run on dozens of incompatible systems: legacy mainframes, modern microservices, partner SaaS, homegrown tools. Each has its own data model, its own identity provider, its own vocabulary. Making them work together is the central challenge of modern enterprise IT — and the one that consumes the most budget without producing visible value.

These three fragmentations are not independent. They compound each other. A system that cannot be integrated cannot be governed. A system that cannot be governed cannot enforce security. A system that cannot enforce security cannot be trusted with intelligent behaviour.

Why the traditional answers fail

Organizations have tried three approaches to these fragmentations. All three are broken.

Centralize. Collapse everything into one monolithic system. This destroys local autonomy, forfeits institutional knowledge, and takes years to deliver. By the time the monolithic system ships, the organization has changed.

Federate loosely. Connect systems through point-to-point adapters. This preserves autonomy but abandons formal guarantees. No unified policy, no unified audit, no way to reason about cross-system behaviour. The adapter count grows as $O(n^2)$ with the number of systems. Every vendor change breaks every adapter.

Rewrite everything. Clean and modern on paper. Bankrupting in practice. No organization has ever successfully completed a wholesale rewrite of its operational systems while continuing to operate them.

What is missing is a framework that *preserves local autonomy while providing global integration guarantees* — one in which each subsystem can retain its own models, its own policies, and its own evolution, while the composition as a whole remains verifiably coherent.

The CZOA insight

Here's the claim that changes everything:

Enterprise systems are a species of intelligent systems — and the same recursive structure that makes them intelligent makes them integrable.

An organisation — with its hierarchical departments, functional roles, operational procedures, and adaptation mechanisms — *instantiates the same structural patterns* that researchers use to model general intelligence. And the very same structure — recursion, containment, composition — is exactly what makes it possible to integrate heterogeneous subsystems without collapsing them.

Look at the parallels:

Intelligence framework	Enterprise system	Integration implication
Hierarchical decomposition	Organisational chart	Composition operator at every level
Component methods	Job functions	Atomic interface for interoperation
Attribute state	Employee data	State is local; only interfaces are shared
Constraint satisfaction	Compliance rules	Policy is inherited from the top down
Adaptive learning	Process improvement	Local models compose without global retraining
Goal optimisation	KPIs	Local objectives scale through hierarchy

These are not metaphors. They are the *same structures* viewed through different lenses. Once you see this, the fragmentation dissolves: an enterprise system already has the structure of an intelligent, integrable system. It just needs to be formalised the same way.

What CZOA gives you

CZOA is the unification. It provides:

1. **A single formalism** (the CZOI 10-tuple) that describes intelligent behaviour, enterprise security, and system integration.
2. **A permission calculus** with formal guarantees, integrated with learning and compositional across subsystems.
3. **A constraint language** (UniLang) that expresses organisational policy formally and applies it hierarchically.
4. **A monitoring architecture** (daemons) that ensures continuous compliance — including cross-subsystem compliance.
5. **A recursive composition** that scales from a single team to a global federation — including legacy systems and merger partners.
6. **A reference implementation** (the CZOI toolkit) that you can use today, with adapter zones for legacy systems and a federation builder for composing independent subsystems.

A taste of the unification

Here is a tiny CZOI system that captures all three dimensions — intelligence, security, and integration:

```

from czo import CZOABuilder, Application, Operation, Role, User, Decision

builder = CZOABuilder("RegionalHealthAuthority")

# ---- Integration perspective: three legacy hospitals, one authority ----
authority = builder.root
hospital_a = builder.add_zone("HospitalA", parent=authority)
hospital_b = builder.add_zone("HospitalB", parent=authority)
hospital_c = builder.add_zone("HospitalC", parent=authority)

# ---- Intelligence perspective: sepsis model in the Emergency zone -----
emergency = builder.add_zone("Emergency", parent=hospital_a, atomic=True)
from czo import Predictor
sepsis_model = Predictor("sepsis", threshold=0.85)
emergency.add_neural("sepsis", sepsis_model)

# ---- Security perspective: unified role taxonomy at the authority -----
app = Application("EMR", zone=authority)
prescribe = app.add_operation(Operation("prescribe"))
authority.add_application(app)

attending = Role("AttendingPhysician", zone=authority,
                 base_permissions=[prescribe])
authority.add_role(attending)

# ---- All three perspectives in one place -----
alice = User("alice", roles={"AttendingPhysician"})
for z in emergency.ancestry():
    z.add_user(alice)

print(builder.permission_engine.decide(alice, prescribe, emergency).name)

```

The sepsis model, the unified permission structure, and the multi-hospital federation all live in **the same zone tree**. They compose. They interact. They evolve. That's CZOA.

Exercises

1. **Observe all three fragmentations.** Think of a system you use daily (email, banking, social media). Identify one "intelligent" feature, one "security" feature, and one integration challenge (a system it talks to that uses a different vocabulary or identity model).
2. **Find the parallels.** Pick an organisation you know. Fill in this table: its hierarchical units, roles, policies, KPIs, and adaptation mechanisms — and for each, how it interacts with systems outside the organization.
3. **Trace the compound effect.** In the merged-hospital scenario above, describe a specific workflow that fails because of all three fragmentations simultaneously. What would "resolution" look like?

Further reading

- The CZOA paper, §1 (Introduction)
- The CZOA paper, §7.7 (Post-merger integration case study)

- Beer's Viable System Model — the pre-CZOA observation that organisations are recursive
- Ross, Weill & Robertson (2006), *Enterprise Architecture as Strategy* — the classical framing of the integration problem

Chapter 2 — Zones: The Recursive Foundation

Learning objectives

- Understand what a zone is and why it's recursive.
- Build zone hierarchies of arbitrary depth.
- Master the containment principle.
- Understand how a zone can also represent a heterogeneous subsystem — including a legacy system.

A zone is a system

In CZOA, **every organisational unit is a zone**, and every zone is a *full* CZOI system. Not a partial one. Not a child in a tree. A complete system with roles, operations, constraints, neural components, and daemons of its own.

Why? Because that's how real organisations work. A hospital's Emergency department isn't just a data structure pointing to its parent. It has its own procedures, its own staff, its own equipment, its own quality metrics. It could be surgically extracted and run as an independent clinic (with some policy adjustments). It is a system.

The same reasoning applies to a legacy system in an integration scenario. A 20-year-old mainframe is not a "subroutine" of the merged organization. It is a full system in its own right — with its own internal logic, its own data, its own operational cadence. Treating it as a first-class subsystem is not an intellectual exercise; it is a precondition for integrating it without breaking it.

CZOA makes this literal. Every zone is a valid CZOI 10-tuple:

```
zone = (Z, R, U, A, O, N, E, Γ, Φ, Δ)
```

The parent-child relationship doesn't merge zones. It **composes** them. The parent governs; the child operates. In an integration scenario, this composition is exactly what makes federation work: the merged authority can impose shared governance at the parent level without intruding into how each legacy system operates internally.

Three kinds of zone

CZOA provides two native zone types plus one integration-specific type:

- **CompositeZone** — a zone that may contain child zones.
- **AtomicZone** — a leaf zone with no children. Calling `add_zone` on it raises a `TypeError`.
- **AdapterZone** — a leaf zone whose internal behaviour is opaque, wrapping a legacy or external system through a declared interface. This is the mechanism by which CZOI integrates systems that

were not built with CZOI in mind.

Building your first tree

```
from czoι import CZOABuilder

builder = CZOABuilder("Company")
root = builder.root

# A composite zone – can have children
engineering = builder.add_zone("Engineering", parent=root)

# Atomic zones – leaves
backend = builder.add_zone("Backend", parent=engineering, atomic=True)
frontend = builder.add_zone("Frontend", parent=engineering, atomic=True)
mobile = builder.add_zone("Mobile", parent=engineering, atomic=True)

print("Tree:")
for zone in root.walk():
    indent = "  " * zone.depth()
    kind = "atomic" if not zone.zones else "composite"
    print(f"{indent}{zone.name} [{kind}]"
```

Output:

```
Tree:
Company [composite]
  Engineering [composite]
    Backend [atomic]
    Frontend [atomic]
    Mobile [atomic]
```

Composite vs atomic

The distinction matters. **AtomicZone** is a promise: this zone will never have children. If someone tries to add a child, they get a **TypeError**:

```
try:
    backend.add_zone(builder.add_zone("Oops", parent=backend))
except TypeError as e:
    print(f"Caught: {e}")
```

Output:

```
Caught: AtomicZone 'Backend' cannot have child zones; use CompositeZone
```

instead

Why enforce this? Two reasons:

1. **Recursion termination.** The paper's definition says the recursion ends when `Z_z = ∅.AtomicZone` makes that explicit.
2. **Accident prevention.** If you know a zone is a leaf, adding a child should be a deliberate design decision, not an accidental line of code.

Rule of thumb: start with `CompositeZone` for anything that might grow, then convert to `AtomicZone` when the design stabilises. Or just use the default (`atomic=False`) until you're sure.

Walking the tree

```
# Pre-order traversal – self, then children, recursively
for zone in root.walk():
    print(zone.name)

# Ancestry – root down to self
for zone in backend.ancestry():
    print(zone.name)

# Depth – root is 0
print(backend.depth())          # 2

# Ancestor check
print(root.is_ancestor_of(backend))    # True
print(backend.is_ancestor_of(root))    # False
```

The containment principle

This is the most important structural rule in CZOA:

$$U_{\text{child}} \subseteq U_{\text{parent}}$$

Every user of a child zone must also be a user of its parent. In English: *if you're in the Backend team, you're automatically in the Engineering department.*

Why does this matter? Because it's how authority flows. A department head can see everyone in their sub-teams — *must* be able to, for governance to work. A company's CEO can see everyone in every department. The user sets are nested.

In an integration scenario, the containment principle is what makes shared governance tractable: the merged authority sees every user in every legacy subsystem, but the legacy subsystems see only their own users. Authority flows downward; autonomy is preserved upward.

The toolkit enforces containment at registration time:


```

from czi import User, ZoneContainmentError

alice = User("alice", roles={"Engineer"})

# This works – backend has engineering as ancestor,
# and we register her at every level
for zone in backend.ancestry():
    zone.add_user(alice)

# But this fails – we try to add a new user to backend only
charlie = User("charlie", roles={"Engineer"})
try:
    backend.add_user(charlie)
except ZoneContainmentError as e:
    print(f"Caught: {e}")

```

Output:

```

Caught: User 'charlie' must be affiliated with parent 'Engineering' first
(containment principle)

```

A useful helper

Registering along the path is so common that you'll want a helper:

```

def register(user, zone):
    """Register `user` at `zone` and every ancestor."""
    for z in zone.ancestry():
        if user.name not in z.users:
            z.add_user(user)

# Now this is one line
register(charlie, backend)

```

We'll use this pattern throughout the tutorial.

Properties: the state of a zone

Every zone has a typed property store:

```

backend.properties.set("headcount", 12, type_hint="int")
backend.properties.set("budget_usd", 1_250_000.0, type_hint="float")
backend.properties.set("on_call_rotation", "weekly", type_hint="str")
backend.properties.set("has_24h_support", True, type_hint="bool")

print(backend.properties.get("headcount"))      # 12

```

```
print(backend.properties.get("missing", 0))      # 0
print(backend.properties.as_dict())
```

Type hints are enforced:

```
try:
    backend.properties.set("headcount", "many", type_hint="int")
except TypeError as e:
    print(f"Caught: {e}")
```

Output:

```
Caught: headcount: expected int, got <class 'str'>
```

Properties are how zones expose state to:

- **Constraints** (e.g., "capacity must not exceed...")
- **Daemons** (e.g., "monitor budget utilisation")
- **Neural components** (e.g., "predict headcount growth")

Deep hierarchies are fine

Don't be afraid of depth. Real organisations have 5–10 levels of hierarchy, and CZOI handles this gracefully.

```
# A realistic geography
builder = CZOABuilder("Global")
na = builder.add_zone("NorthAmerica", parent=builder.root)
us = builder.add_zone("US", parent=na)
west = builder.add_zone("WestCoast", parent=us)
bay = builder.add_zone("BayArea", parent=west)
sf = builder.add_zone("SanFrancisco", parent=bay, atomic=True)
oak = builder.add_zone("Oakland", parent=bay, atomic=True)

print(f"sf depth: {sf.depth()}")      # 5
print(f"sf ancestry: {[z.name for z in sf.ancestry()]}")
# ['Global', 'NorthAmerica', 'US', 'WestCoast', 'BayArea', 'SanFrancisco']
```

Six levels deep. No problem. The recursion is what makes CZOA scale.

Common pitfalls

Pitfall 1: trying to add children to an atomic zone.

```
leaf = builder.add_zone("Leaf", atomic=True)
```

```
# leaf.add_zone(...) # TypeError
```

If you need children, don't mark the zone atomic.

Pitfall 2: forgetting containment.

```
# Wrong: jumps straight to the leaf
sf.add_user(alice) # ZoneContainmentError

# Right: walk the path
register(alice, sf)
```

Pitfall 3: assuming children auto-inherit users.

Children don't inherit users. They can *only* hold users who are already at the parent. The set is a subset, not an automatic copy.

```
# After registering alice at every level:
print("alice" in sf.users)           # True
print("alice" in bay.users)          # True
print("alice" in west.users)         # True
```

But if you add a new user to a child, you must add them to the parent first.

Exercises

1. **Model a university.** Build a hierarchy: University → Colleges → Departments → Research Groups. Use **CompositeZone** for anything with children, **AtomicZone** for leaves. Print the tree.
2. **Containment violation.** Deliberately trigger a **ZoneContainmentError**. Fix it. Then deliberately trigger a **TypeError** from an atomic zone. Fix it.
3. **Properties.** Add a **capacity** property to each zone in your university. Enforce **int**. Try to set it to a string and observe the error.
4. **Deep recursion.** Build a 10-level hierarchy. Register a user at the leaf. Verify they're in every ancestor.
5. **Sketch an adapter.** Imagine a legacy student information system. Write down three operations it would expose, one role it would define, and one property it would publish — without writing any code yet.

Further reading

- The CZOA paper, Definition 1 (Recursive CZOA)
- The CZOA paper, Definition 5 (Adapter Zone)
- The User Guide, §5 (Building Zone Hierarchies)

Chapter 3 — The 10-Tuple

Learning objectives

- Understand every component of the CZOI 10-tuple.
- See why minimality matters — and how it enables clean integration.
- Recognise each component in a real system, including a federated one.

The tuple

A CZOI system is:

$$S = (Z, R, U, A, O, N, E, \Gamma, \Phi, \Delta)$$

Ten components. Let's go through each one carefully — and in each case, note how the component contributes to *integration*, not just to security or intelligence.

Z — Zones

The **subsystems**. Every zone has a **zones** dict:

```
builder = CZOABuilder("Org")
hr = builder.add_zone("HR", parent=builder.root)
it = builder.add_zone("IT", parent=builder.root)

print(builder.root.zones)           # {'HR': ..., 'IT': ...}
```

In an integration scenario, the child zones may include **AdapterZone** instances wrapping legacy systems. From the parent's perspective, there is no difference: an adapter zone participates in the recursion exactly like a native zone.

R — Roles

The **job functions**. Each role belongs to a zone and has base permissions:

```
from czoitutorial import Role, Operation, Application

app = Application("Payroll", zone=builder.root)
process = app.add_operation(Operation("process_payroll"))
view = app.add_operation(Operation("view_payroll"))
builder.root.add_application(app)

admin = Role("PayrollAdmin", zone=builder.root,
             base_permissions=[process, view])
viewer = Role("PayrollViewer", zone=builder.root,
              base_permissions=[view])
admin.add_junior(viewer)
```

```
builder.root.add_role(admin)
builder.root.add_role(viewer)
```

Roles have: a name, a zone of definition, a set of base operations, a set of junior roles (seniority), and a property store for attributes. When two subsystems are federated, their role taxonomies remain distinct — but semantic embeddings can discover equivalence classes across them (Chapter 8).

U — Users

The **identities**. Each user holds role names and has attributes:

```
from czoit import User

alice = User(
    "alice",
    roles={"PayrollAdmin"},
    attributes={"employee_id": "E-1", "clearance": 3},
    credentials={"mfa_enabled": True},
)
builder.root.add_user(alice)
```

Roles on a user are **names**, not objects. The zone resolves them. This is what lets a user carry their role through nested zones — and what makes cross-system identity federation tractable: each subsystem declares how it maps federated identities to its local roles.

A — Applications

The **structural modules**. Applications group operations:

```
from czoit import Application

hr_app = Application("HRApp", zone=builder.root)
hire = hr_app.add_operation(Operation("hire_employee"))
fire = hr_app.add_operation(Operation("terminate_employee"))
builder.root.add_application(hr_app)
```

Applications are: **not** permission targets; deployable units; grouping for UI, embedding, and auditing. In an integration scenario, an application name often reveals its origin — `LegacyHR.view_employee` is obviously from the legacy system, and the embedding alignment functor uses that signal to help discover equivalences.

O — Operations

The **atomic executable actions**. This is where permissions live.

```
print(hire.qualified_name)    # "HRAApp.hire_employee"
print(hire.application.name)  # "HRAApp"
print(hire.name)             # "hire_employee"
```

Every permission is granted on an operation:

```
admin_role.grant(hire)        # OK
# admin_role.grant(hr_app)    # WRONG – apps are not permission targets
```

Why operations and not applications? Because an application like **HRAApp** might expose 50 different operations, and you want to grant a role *some* of them, not all. In integration, the operation is the atomic unit of the interface contract: two subsystems are integrable at the operation level whenever their operations can be mapped to each other (exactly, or semantically).

N — Neural components

The **learnable functions**. Any object with a **predict** method works:

```
from czoitutorial import Predictor
import numpy as np

model = Predictor("attrition", threshold=0.5)
X = np.array([[1, 0.9, 5], [0, 0.3, 1], [0, 0.2, 2], [1, 0.8, 4]])
y = np.array([1.0, 0.0, 0.0, 1.0])
model.fit(X, y, epochs=500)

builder.root.add_neural("attrition", model)
```

Neural components live **inside zones** — this is critical. Different zones can have different models:

```
backend.add_neural("perf_model", backend_model)
frontend.add_neural("perf_model", frontend_model)
```

Each zone decides what to learn. In a federation, this is exactly what preserves local intelligence: each legacy subsystem keeps its own models, trained on its own data, without any requirement to share them or re-train them globally. The federation composes their outputs, not their state.

E — Embeddings

The **semantic vectors**. They map entities into a shared space where similarity is geometric distance.

```
from czoitutorial import EmbeddingService
```

```
emb = EmbeddingService(dimension=64)
v_hire = emb.embed_operation(hire)
v_fire = emb.embed_operation(fire)
v_process = emb.embed_operation(process)

print(f"hire ↔ fire: {emb.similarity(v_hire, v_fire):.3f}")
print(f"hire ↔ process: {emb.similarity(v_hire, v_process):.3f}")
```

Embeddings enable: **cross-zone role matching, anomaly detection, adaptive grants**. In integration, embeddings are the mechanism by which heterogeneous vocabularies are bridged: two subsystems can name the same operation differently, and the aligned embeddings still reveal the equivalence.

Γ — Constraints

The **policy system**, split into four families:

- **I** — Identity: invariants that always hold.
- **T** — Trigger: event-condition-action rules.
- **G** — Goal: optimisation objectives.
- **C** — Access: permission policies.

```
builder.add_access_constraint("""
    signature {
        sort User;
        predicate approve(u: User);
        predicate execute(u: User);
    }
    forall u: User . not (approve(u) and execute(u))
""")
```

This is a separation-of-duty rule expressed formally. In a federation, constraints are **hierarchical**: the parent authority can impose shared constraints that apply to every subsystem, while each subsystem can add stricter local rules. This is the formal mechanism by which shared governance is imposed on independent actors without violating their internal policies.

Φ — Permission calculus

The **decision function**. It's two-stage recursive:

1. **Local** — check the user's roles in this zone.
2. **Parent override** — if inconclusive, recurse to the parent.

```
from czoitutorial import Decision

decision = builder.permission_engine.decide(alice, hire, builder.root)
print(decision.name)    # ALLOW / DENY / INCONCLUSIVE
```

We'll cover Φ in depth in Chapter 6. In a federation, this recursive structure is what guarantees that cross-zone access passes through both the requesting and the target subsystem's local decision functions — the more restrictive constraint wins.

Δ — Daemons

The **continuous monitors**. Daemons observe the system and emit signals.

```
from czoim import Daemon, DaemonSignal

class AttritionDaemon(Daemon):
    def __init__(self, model, parent=None):
        super().__init__("AttritionDaemon", parent=parent, interval=5.0)
        self.model = model

    def monitor(self):
        score = self.model.predict({"tenure": 2, "satisfaction": 0.3,
                                    "promotions": 0})

        if score > 0.8:
            self.emit_signal(
                DaemonSignal.STATE_WARNING,
                {"risk": score, "reason": "attrition"},
            )

daemon = AttritionDaemon(model)
builder.add_daemon(daemon)
```

We'll cover daemons fully in Chapter 11. In a federation, **cross-zone daemons** observe pairs of subsystems and emit into their common ancestor — the mechanism for detecting integration issues before they become incidents.

Why exactly these ten?

The ten components are **minimal** and **orthogonal**:

- **Minimal**: remove any one and the framework can't express something important. Without Z , no hierarchy. Without Φ , no permission decisions. Without Γ , no policy. Without Δ , no monitoring.
- **Orthogonal**: each component has a distinct role; none overlaps another. R and O are different (roles are job functions, operations are actions). N and E are different (neural components learn, embeddings represent). Γ and Δ are different (constraints are declarative, daemons are procedural).

This orthogonality is what makes CZOI **composable** — and, therefore, what makes it **integrable**. If two components did the same job, you'd have to decide which one to use in each context, and the system would have internal ambiguity that breaks under composition.

The tuple in one example

```
from czoim import (
    Application, CZOABuilder, Daemon, DaemonSignal, Operation,
```



```

    Predictor, Role, User,
)

# Z – zones
builder = CZOABuilder("Team")
team = builder.add_zone("Engineering", parent=builder.root)
backend = builder.add_zone("Backend", parent=team, atomic=True)

# A – applications
app = Application("Repo", zone=team)
commit = app.add_operation(Operation("commit"))
review = app.add_operation(Operation("review"))
team.add_application(app)

# R – roles
senior = Role("Senior", zone=team, base_permissions=[commit, review])
junior = Role("Junior", zone=team, base_permissions=[commit])
senior.add_junior(junior)
team.add_role(senior)
team.add_role(junior)

# U – users
alice = User("alice", roles={"Senior"})
bob = User("bob", roles={"Junior"})
for z in backend.ancestry():
    z.add_user(alice)
    z.add_user(bob)

# N – neural
review_model = Predictor("review_time", threshold=0.5)
review_model.fit(
    X=[[1, 5, 0], [0.5, 20, 1], [1, 3, 0], [0.3, 40, 1]],
    y=[0.0, 1.0, 0.0, 1.0],
    epochs=500,
)
team.add_neural("review_time", review_model)

# E – embeddings
team.set_embeddings(builder.embedding_service)

# Γ – constraints
builder.add_access_constraint("""
    signature {
        sort User;
        predicate hasSenior(u: User);
        predicate selfReview(u: User);
    }
    forall u: User . not (hasSenior(u) and selfReview(u))
""")

# Φ – permission engine (attached by builder)
engine = builder.permission_engine

# Δ – daemons

```

```

class RepoDaemon(Daemon):
    def monitor(self):
        pass
builder.add_daemon(RepoDaemon("Repo"))

# Everything's in place. Use it.
print(engine.decide(alice, review, team).name) # ALLOW
print(engine.decide(bob, review, team).name)   # INCONCLUSIVE

```

Ten components, ten lines of setup, one complete system.

Exercises

1. **Identify the components.** In the example above, point to where each of the ten components appears. Write it out as a list.
2. **Add a component.** Add a `view_repo` operation. Grant it to both `Senior` and `Junior`. Verify that both can view, but only `Senior` can review.
3. **Extend the neural model.** Add a second model to the `backend` zone that predicts something different. Show that both models coexist.
4. **Minimality test.** Try to design a system that would need an eleventh component. What would it be? Can you express it using the existing ten?
5. **Integration angle.** For each of the ten components, write one sentence explaining how it contributes to integrating a heterogeneous subsystem. If any component seems irrelevant, revisit Chapter 1.

Further reading

- The CZOA paper, Definition 1 and §3
- Chapter 4 (Roles, users, and permissions in depth)

Part II — Core Mechanics

Chapter 4 — Roles, Users, and Seniority

Learning objectives

- Design role hierarchies that mirror real job families.
- Understand the difference between base and effective permissions.
- Use seniority to avoid permission duplication.

The role model

A **role** is a named job function within a zone. It has a name, a base permission set, a junior role set (seniority), and a property store. Roles form a partial order: $r_1 \geq_Z r_2$ means r_1 inherits r_2 's permissions.

```

from czoitutorial import Role, Operation, Application, CZOABuilder

builder = CZOABuilder("Bank")

```

```

bank = builder.root

app = Application("CoreBanking", zone=bank)
deposit = app.add_operation(Operation("deposit"))
withdraw = app.add_operation(Operation("withdraw"))
approve_loan = app.add_operation(Operation("approve_loan"))
view_transactions = app.add_operation(Operation("view_transactions"))
bank.add_application(app)

# A realistic role family
teller = Role("Teller", zone=bank,
              base_permissions=[deposit, withdraw, view_transactions])
senior_teller = Role("SeniorTeller", zone=bank,
                     base_permissions=[approve_loan])
manager = Role("BranchManager", zone=bank,
               base_permissions=[]) # no extra base permissions

# Seniority chain: Manager is senior to SeniorTeller is senior to Teller
manager.add_junior(senior_teller)
senior_teller.add_junior(teller)

bank.add_role(teller)
bank.add_role(senior_teller)
bank.add_role(manager)

```

The **base permissions** of a role are what you explicitly grant. The **effective permissions** include everything inherited from junior roles. The Manager has *no* base permissions of their own but inherits everything because they're senior to SeniorTeller, who is senior to Teller.

Role taxonomies and integration

When two subsystems come from different origins — a merger, an acquisition, a partner API — their role taxonomies are typically distinct. CZOI preserves both: each subsystem keeps its own roles internally, and the federation discovers semantic equivalence between roles across subsystems using the embedding alignment functor. Chapter 15 shows how this works in practice.

Users hold role names, not role objects

A user's **roles** attribute holds **role names**. When the engine checks **alice** in a zone, it looks up the name in *that zone's* **roles** dict. This is what allows a user to carry their role through nested zones — and what makes cross-system identity federation tractable.

```

alice = User("alice", roles={"SeniorTeller"})

branch = builder.add_zone("MainBranch", parent=bank, atomic=True)
for z in branch.ancestry():
    z.add_user(alice)

# Both zones have "SeniorTeller" (inherited from parent)
print("SeniorTeller" in bank.roles) # True

```

```
print("SeniorTeller" in branch.roles)      # True
print(bank.roles["SeniorTeller"] is branch.roles["SeniorTeller"])  # True
```

Exercises

- Seniority is a partial order between roles in the same zone.
- Effective permissions = base permissions + everything inherited from junior roles.
- In a federation, seniority is local to each subsystem — no cross-zone seniority is implied.
- **Exercise:** Design a four-level role hierarchy for a support team; verify that the top role inherits everything.
- **Exercise:** Add a cross-branch role that shares permissions with two other roles via a common junior ancestor.

Further reading

- The CZOA paper, §3, item 2 (Roles)
- The User Guide, §6 (Roles, Users, and Operations)

Chapter 5 — Operations and the Application Boundary

Learning objectives

- Understand why applications and operations are separate.
- Design operations at the right granularity.
- Use applications for organisation, not for permission.

An **application** is a structural module — a deployable unit that groups related operations. An **operation** is an atomic permission target. In CZOI, you grant permissions on *operations*, never on applications. This is what keeps the framework minimal, granular, auditable, and — crucially for integration — what makes operations the atomic interface contract between subsystems.

```
app = Application("Payroll", zone=hr)
view  = app.add_operation(Operation("view_payroll"))
edit  = app.add_operation(Operation("edit_payroll"))
process = app.add_operation(Operation("process_payroll"))
approve = app.add_operation(Operation("approve_payroll"))
```

Grant them individually to preserve least privilege:

```
auditor_role.grant(view)
hr_admin_role.grant(view)
hr_admin_role.grant(edit)
hr_admin_role.grant(process)
controller_role.grant(view)
controller_role.grant(approve)
```

Operations as integration contracts

Two subsystems are integrable at the operation level whenever their operations can be mapped to each other — exactly or semantically. This is why CZOI insists on operations as the permission target: it gives every subsystem a well-defined interface at the atomic level, and it lets the federation discover correspondences between subsystems whose applications, role names, and vocabularies all differ.

Exercises

- Applications are deployable units; operations are permission targets.
- Four questions decide whether an operation should be split or merged.
- In integration, operations are the atomic interface contract.
- **Exercise:** Model a helpdesk system's operations, justifying each split.

Further reading

- The CZOA paper, §3, items 4 and 5 (Applications and Operations)
- The CZOA paper, §8.4 (Design rationale)

Chapter 6 — The Permission Calculus

Learning objectives

- Understand the two-stage recursive decision function.
- Know when a decision is **ALLOW**, **DENY**, or **INCONCLUSIVE**.
- Trace a decision through multiple zones.

The permission calculus Φ is a two-stage recursive decision function:

1. **Local decision.** Look at the user's roles in this zone.
2. **Parent override.** If the local decision is inconclusive, recurse to the parent.

The decision is three-valued:

Value	Meaning
ALLOW	A covering role was found and constraints passed.
DENY	Either a covering role was found but a constraint failed, or the recursion reached the root with no covering role.
INCONCLUSIVE	This zone's roles don't cover the operation, but a parent might. Only returned by <code>evaluate_local</code> .

```
engine = builder.permission_engine

# All three decision values are semantically distinct
print(engine.decide(alice, prescribe, icu).name)    # ALLOW
```

```
print(engine.decide(bob, prescribe, icu).name) # DENY
print(engine.evaluate_local(bob, prescribe, icu).name) # INCONCLUSIVE
```

Cross-zone decisions

In a federation, a decision for a user in subsystem A requesting an operation in subsystem B traverses both subsystems' local decision functions. The audit record shows the full path — including which subsystem imposed which constraint. This is what makes cross-subsystem compliance *auditable*, not just observable.

Exercises

- Decision is two-stage: local, then parent override.
- **ALLOW**, **DENY**, **INCONCLUSIVE** are semantically distinct.
- The cache is invalidated automatically on grants and revocations.
- In a federation, the more restrictive constraint wins across subsystem boundaries.
- **Exercise:** Trace **bob.dispense** through a multi-zone hierarchy; print every intermediate decision.

Further reading

- The CZOA paper, §3, item 9 (Permission Calculus)
- The User Guide, §7 (The Permission Calculus)

Part III — Intelligence

Chapter 7 — Neural Components

Learning objectives

- Understand why neural components are first-class citizens in CZOI.
- Train a **Predictor**, **AnomalyDetector**, and **RoleMiner**.
- Attach neural components to zones and use them in daemons and permission hooks.

The toolkit ships with three neural primitives plus a wrapper for arbitrary models:

- **Predictor** — a trainable linear model with sigmoid output.
- **AnomalyDetector** — an autoencoder for detecting outliers.
- **RoleMiner** — unsupervised discovery of role structures.

```
from czoitutorial import Predictor, AnomalyDetector, RoleMiner

# Predictor
model = Predictor("sepsis", threshold=0.85)
model.fit(X_train, y_train, epochs=500)
model.fires({"hr": 0.9, "temp": 0.8, "lactate": 0.9})

# AnomalyDetector
detector = AnomalyDetector("access", input_dim=4, latent_dim=2)
```

```

detector.fit(X_normal, epochs=300)
detector.is_anomalous(x)

# RoleMiner
miner = RoleMiner(latent_dim=4, min_cluster_size=2)
result = miner.mine(X_binary, operation_names)

```

Local intelligence in federations

In a federation, neural components are *strictly local*. Each subsystem keeps its own models, trained on its own data, without any requirement to share them or re-train them globally. The federation composes their *outputs*, not their *state*. This is what preserves local autonomy while enabling shared governance — each subsystem's intelligence remains its own, even as the federation's policy applies to all.

Exercises

- Neural components live inside zones — each subsystem keeps its own models.
- Neural components supplement constraints; they never replace them.
- **Exercise:** Train a **Predictor** for "will miss deadline"; attach it to a **Projects** zone; write a daemon that flags projects above 0.7.
- **Exercise:** Use **RoleMiner** on a synthetic access matrix with 3 clear groups. Verify it discovers the groups.

Further reading

- The CZOA paper, §5 (Learning-enhanced access control)
- The User Guide, §10 (Neural Components)

Chapter 8 — Semantic Embeddings

Learning objectives

- Understand what embeddings are and why they matter for cross-zone reasoning.
- Train an alignment layer with contrastive loss.
- Use embeddings for cross-zone role matching and anomaly detection.

Embeddings turn "similarity" into geometry: two entities are similar if their vectors are close. The **EmbeddingService** maps operations, roles, and zones into a Hilbert space where similarity is geometric.

```

from czo import EmbeddingService

emb = EmbeddingService(dimension=64)

v1 = emb.embed_operation(op1)
v2 = emb.embed_operation(op2)
print(emb.similarity(v1, v2))

```

```
# Contrastive training of the global alignment functor
emb.train_alignment(positives=[(a, b)], negatives=[(a, c)])
```

The alignment functor is the integration mechanism

When two subsystems come from different origins — a merger, an acquisition, a partner API — they use different vocabulary for the same concepts. The alignment functor projects both subsystems' local embeddings into a shared Hilbert space, revealing semantic equivalence. This is the technical mechanism by which CZOI integrates heterogeneous vocabularies: even if two subsystems name the same operation differently, the aligned embeddings still reveal the equivalence. In the paper's post-merger case study, this mechanism discovered **47 semantically equivalent role pairs** across two legacy taxonomies — a task that would otherwise have consumed several weeks of manual effort.

Exercises

- Embeddings turn similarity into geometry.
- The alignment functor bridges heterogeneous vocabularies across subsystems.
- In a post-merger scenario, alignment discovers semantically equivalent roles automatically.
- **Exercise:** Train an alignment on 20 domain-specific positive pairs and 20 negative pairs. Verify positive pairs become more similar.

Further reading

- The CZOA paper, §5.2 (Semantic embeddings)
- The User Guide, §11 (Semantic Embeddings)

Chapter 9 — Adaptive Access Control

Learning objectives

- Understand when and why permissions should adapt.
- Implement adaptive grants and revocations.
- Use neural contributions for context-aware decisions.
- Preserve safety while adapting.

During a surge — a flu outbreak, a market crash, a supply-chain disruption — the system may need to grant someone a permission they don't normally have. CZOI provides two mechanisms:

1. **Dynamic grants** — change a role's base permissions at runtime via `zone.grant` / `zone.revoke`.
2. **Neural contributions** — adjust individual decisions via a hook.

```
from czi import NeuralContribution

def surge_hook(user, operation, zone, base_decision):
    if (operation.name == "prescribe"
        and user.has_role("SeniorNurse")
        and zone.properties.get("surge_active", False)):
        return Decision.ALLOW
```



```

    return base_decision

    builder.permission_engine.set_neural_contribution(
        NeuralContribution(surge_hook)
    )

```

Theorem 7 of the paper guarantees that any adaptive update which preserves monotonicity, satisfies all constraints, and maintains a complete audit trail is safe.

Adaptation in federations

In a federation, an adaptive update at any subsystem preserves the composed system's safety *if and only if* it respects the shared constraints established by the cross-zone integration relation. This follows directly from the Integration Theorem: any cross-zone operation passes through both subsystems' local decision functions, and the more restrictive constraint wins.

Exercises

- Adaptation is a first-class operation, subject to safety guarantees.
- Use grants/revocations for well-defined events; hooks for context-sensitive decisions.
- In a federation, adaptive updates must respect shared constraints.
- **Exercise:** Build a hospital surge example where a daemon grants **prescribe** to senior nurses; verify the decision changes.

Further reading

- The CZOA paper, §5.3 (Adaptive access control)
- The CZOA paper, Theorem 7 (Safety of adaptation)
- The User Guide, §12 (Adaptive Access Control)

Part IV — Governance

Chapter 10 — Constraints via UniLog

Learning objectives

- Understand the four families of constraints (I, T, G, C).
- Write UniLang constraints for real organisational policies.
- Register custom predicates.
- Understand how constraints interact with permissions.

CZOA defines four families of constraints — $\Gamma = (\mathbf{I}, \mathbf{T}, \mathbf{G}, \mathbf{C})$ — expressed in UniLang, a first-order logic with temporal and modal extensions:

Family	Name	Purpose
I	Identity	Invariants that must always hold

Family	Name	Purpose
T	Trigger	Event-condition-action rules
G	Goal	Optimisation objectives
C	Access	Permission-related policies

```
builder.add_access_constraint("""
    signature {
        sort User;
        predicate prescribe(u: User);
        predicate dispense(u: User);
    }
    forall u: User . not (prescribe(u) and dispense(u))
    """)
```

Access constraints have **veto power** over role-based permissions: a user with the covering role can still be denied if a constraint fails.

Shared constraints in a federation

When subsystems are federated, the parent can impose **shared constraints** that apply to every subsystem. A child can add stricter local constraints, but cannot weaken a parent's. This is the formal mechanism by which shared governance is imposed on independent actors without violating their internal policies — and it is a one-line operation on the federation builder:

```
fed.share_constraint("""... UniLang ...""")
```

Exercises

- Four families: Identity (I), Trigger (T), Goal (G), Access (C).
- Constraints are declarative, verifiable, portable, and readable.
- Constraints are hierarchical: parents can impose shared rules on all subsystems.
- **Exercise:** Write a SoD constraint that prevents any user from both `submit_expense` and `approve_expense`.

Further reading

- The CZOA paper, §3, item 8 (Constraint System)
- The UniLog toolkit documentation
- The User Guide, §8 (Constraints with UniLog)

Chapter 11 — Daemons and Continuous Monitoring

Learning objectives

- Understand what daemons are and how they differ from permission checks.
- Build hierarchical daemon trees.
- Use signals for coordination.
- Handle errors and lifecycle.

Permission checks happen on demand. Daemons run **continuously**, reading system state and emitting typed signals up a daemon tree. Signals propagate upward; `on_signal` handles them at each level; a root daemon aggregates and can act.

```
from czoitutorial import Daemon, DaemonSignal

class BatteryDaemon(Daemon):
    def __init__(self, robots, threshold=0.2, parent=None):
        super().__init__("BatteryDaemon", parent=parent, interval=1.0)
        self.robots = robots
        self.threshold = threshold

    def monitor(self):
        for r in self.robots:
            if r.attributes.get("battery", 1.0) < self.threshold:
                self.emit_signal(DaemonSignal.STATE_WARNING,
                                {"robot": r.name})
```

Cross-zone daemons

In a federation, **cross-zone daemons** observe pairs of subsystems and emit into their common ancestor. They detect integration issues before they become incidents: vocabulary drift (a subsystem's roles gradually diverge from the shared taxonomy), latency spikes in cross-zone flows, and anomalous interaction patterns. Chapter 15 shows how to wire these up.

Exercises

- Daemons are continuous monitors that emit typed signals up a daemon tree.
- Signals propagate upward; `on_signal` handles them at each level.
- In a federation, cross-zone daemons observe subsystem interactions.
- **Exercise:** Build a warning daemon that fires when failed login attempts exceed a threshold.

Further reading

- The CZOA paper, §5.4 (Constraint daemons)
- The User Guide, §9 (Daemons and Signals)

Part V — Real Systems

Chapter 12 — Designing for a Domain

Learning objectives

- Translate an organisation into a CZOI zone tree.
- Choose the right level of abstraction.
- Identify roles, operations, and constraints.
- Anticipate where adaptation will be needed.

CZOA is not just a runtime — it's a design methodology. The ten-step process is:

1. **Map the organisation.** Draw the reporting structure.
2. **Identify the zone tree.** Choose which units become zones.
3. **Define roles.** Enumerate job functions within each zone.
4. **Enumerate operations.** List the atomic actions each zone supports.
5. **Grant base permissions.** Assign operations to roles.
6. **Write constraints.** Capture policy invariants.
7. **Identify adaptation points.** Where will conditions change?
8. **Choose neural components.** Which zones benefit from learning?
9. **Design daemons.** What must be monitored?
10. **Iterate.** Validate with domain experts.

Apply this to a regional hospital network. The design matrix captures the output: zone × roles × operations × constraints × daemons × neural.

Design for integration from day one

When designing a system that will eventually be federated with others — through a merger, a partnership, or a platform consolidation — keep the **integration interface** in mind at every design step. Make operation names semantically meaningful (so the alignment functor can find correspondences later). Keep role taxonomies at the level where they're shared. Put constraints that must survive integration at the highest zone where they apply. These are cheap decisions made early, and expensive ones made late.

Exercises

- Ten-step process: map, model, enumerate, grant, constrain, adapt, learn, monitor, iterate.
- The design matrix captures the output: zone × roles × operations × constraints × daemons × neural.
- **Exercise:** Design a CZOI model for a regional bank with branches, roles, and constraints. Include at least 3 levels of hierarchy.

Further reading

- The CZOA paper, §7 (Evaluation)
- The User Guide, §16 (Complete Worked Examples)

Chapter 13 — Testing and Verification

Learning objectives

- Write unit tests for CZOI systems.
- Verify permission decisions, constraint satisfaction, and adaptive behaviour.

Every permission decision is a testable assertion. Test **ALLOW**, **DENY**, and **INCONCLUSIVE** separately — they mean different things. Verify cache invalidation on grants and revocations. Test constraints explicitly — a test that only exercises **ALLOW** doesn't prove the **DENY** path works.

```
def test_attending_can_prescribe(hospital):
    result = hospital["builder"].permission_engine.decide(
        hospital["alice"], hospital["prescribe"], hospital["emergency"],
    )
    assert result is Decision.ALLOW

def test_nurse_cannot_prescribe(hospital):
    result = hospital["builder"].permission_engine.decide(
        hospital["bob"], hospital["prescribe"], hospital["emergency"],
    )
    assert result is Decision.DENY
```

Testing federations

For federations, add tests that exercise cross-subsystem decisions in both directions. Verify that shared constraints at the parent level fire for users in each subsystem. Verify that a subsystem's local constraint is still enforced after federation. And run `verify_integration(fed)` to check that the composed system is soundly integrated.

Exercises

- Every permission decision is a testable assertion.
- Test **ALLOW**, **DENY**, and **INCONCLUSIVE** separately.
- Verify cache invalidation on grants and revocations.
- In a federation, test cross-zone decisions from both subsystems.
- **Exercise:** Write a test suite for the hospital example covering all roles and operations.

Further reading

- The CZOI toolkit test suite (in the repository)
- The User Guide, §13 (Audit and Observability)

Chapter 14 — Deployment Patterns

Learning objectives

- Choose a deployment architecture for CZOI.
- Integrate with existing systems.
- Handle persistence, scaling, and observability.
- Plan for updates and migrations.

Where does the CZOI runtime live? Four patterns:

1. **Embedded** — inside your application process.

2. **Sidecar** — a separate service alongside your app.
3. **Centralised** — one shared permission service.
4. **Hybrid** — per-zone runtimes coordinated globally (recommended for production).

In a federation, each subsystem can use its own deployment pattern. The federation itself typically runs a hybrid pattern: each subsystem runs its own local CZOI runtime, and the parent coordinates policy through a shared policy service.

Exercises

- Embedded for simplicity; sidecar for separation; centralised for unified policy; hybrid for scale.
- Persistence is a separate concern: configuration, user assignments, audit records, model state, adaptive state.
- **Exercise:** Design a database schema for storing zones, roles, and user assignments; write the load function.

Further reading

- The User Guide, §15 (Persistence and Web Frameworks)

Chapter 15 — Integrating Heterogeneous Systems

Learning objectives

- Understand the three integration patterns (parallel, alternative, governed) and when to use each.
- Wrap a legacy system as an adapter zone without modifying it.
- Federate independent subsystems under a shared governance parent.
- Discover semantically equivalent roles across subsystems automatically.
- Verify integration soundness using the Integration Theorem.

The problem, restated

Real organizations run on dozens of incompatible systems. An insurance company has one core policy admin system from the 1990s, a claims system from the 2000s, a modern customer portal, and a partner API. A hospital network formed from a merger has three legacy EMRs, each with its own role taxonomy. A bank that acquired a fintech has two completely different identity providers, two different regulatory regimes, and two different approaches to separation of duty.

Traditional integration approaches fall into three categories, all unsatisfying:

- **Centralize** — collapse everything into a single system, destroying autonomy.
- **Federate loosely** — connect systems through adapters, abandoning formal guarantees.
- **Rewrite** — clean on paper, bankrupting in practice.

CZOA offers a fourth path: **recursive federation**. Each subsystem — legacy or modern — becomes a zone. Zones compose under shared governance. The composition is a formal operation with verifiable properties. Local autonomy is preserved by construction.

Adapter zones: the mechanism for integrating legacy systems

An **adapter zone** is a leaf CZOI subsystem whose internal behaviour is opaque, but whose *interface* — a set of roles, operations, and constraints — is declared explicitly. It is the mechanism by which CZOI treats a legacy system as a first-class subsystem without modifying the legacy system itself.

```
from czoitutorial import AdapterZone, Operation, Role

# The adapter wraps the legacy system. It implements two methods:
#   execute(operation_name, user_name, kwargs) -> result
#   describe_interface() -> {operations, roles, properties}
class LegacyHRAdapter:
    def __init__(self, connection_string):
        self.conn = legacy_connect(connection_string)

    def execute(self, operation_name, user_name, kwargs):
        # Translate the CZOI operation into a legacy call.
        if operation_name == "LegacyHR.view_employee":
            return self.conn.read_employee(kwargs["employee_id"])
        elif operation_name == "LegacyHR.edit_employee":
            return self.conn.update_employee(
                kwargs["employee_id"], kwargs["changes"],
            )

# Wrap the legacy system as a CZOI zone.
legacy_hr = AdapterZone(
    name="LegacyHR",
    adapter=LegacyHRAdapter("sql://legacy-hr.internal"),
)

# Declare the CZOI-facing interface.
view_employee = Operation("view_employee")
edit_employee = Operation("edit_employee")
legacy_hr.expose_operations({
    "view_employee": view_employee,
    "edit_employee": edit_employee,
})

hr_viewer = Role("HRViewer", zone=legacy_hr,
                 base_permissions=[view_employee])
hr_editor = Role("HREditor", zone=legacy_hr,
                 base_permissions=[view_employee, edit_employee])
hr_editor.add_junior(hr_viewer)
legacy_hr.add_role(hr_viewer)
legacy_hr.add_role(hr_editor)
```

From the federation's perspective, **legacy_hr** is indistinguishable from a natively built zone. Its internal state is opaque; only its declared interface participates in permission checks, constraint evaluation, and audit. **The legacy system has not been modified in any way.**

Federating independent subsystems

Once each subsystem — legacy or native — is exposed as a zone, federation is a composition operation.

```

from czoim import FederationBuilder

fed = FederationBuilder("MergedHealthAuthority")

# Adopt each subsystem under a common parent.
fed.adopt_subsystem(legacy_hr_a, name="LegacyRegionA")
fed.adopt_subsystem(legacy_hr_b, name="LegacyRegionB")
fed.adopt_subsystem(shared_services, name="SharedServices")

# Impose shared governance at the parent level.
fed.share_constraint("""
    signature {
        sort User, Role;
        constant HRViewer : Role;
        constant FinanceApprover : Role;
        predicate hasRole(u: User, r: Role);
    }
    forall u: User .
        not (hasRole(u, HRViewer) and hasRole(u, FinanceApprover))
""")

# Wire cross-zone monitoring.
fed.add_cross_zone_daemon(FederationHealthDaemon())

# The federation is a full CZOI system.
engine = fed.permission_engine
print(engine.decide(alice, view_employee, legacy_hr_a).name)

```

The federation is a CZOI system. It has a permission engine, a constraint manager, a daemon manager, and embeddings — all shared with its subsystems, exactly as in a native CZOI tree.

Discovering role equivalence across subsystems

In a merger, the two legacy systems likely have semantically equivalent roles under different names. The alignment functor discovers these automatically.

```

def discover_role_equivalences(zone_a, zone_b, threshold=0.75):
    """Find semantically equivalent roles across two subsystems."""
    equivalences = []
    for role_a in zone_a.roles.values():
        vec_a = emb.embed_role(role_a)
        for role_b in zone_b.roles.values():
            vec_b = emb.embed_role(role_b)
            sim = emb.similarity(vec_a, vec_b)
            if sim > threshold:
                equivalences.append((role_a, role_b, sim))
    return sorted(equivalences, key=lambda x: -x[2])

# Find equivalent roles across the two legacy regions.
equivalences = discover_role_equivalences(legacy_hr_a, legacy_hr_b)

```



```
for role_a, role_b, sim in equivalences[:5]:
    print(f"{role_a.name} ↔ {role_b.name}: {sim:.3f}")
```

In the paper's post-merger case study, this mechanism discovered **47 semantically equivalent role pairs** across two legacy taxonomies — a task that would otherwise have consumed several weeks of manual effort.

The discovered equivalences are not automatically applied — a human reviewer approves them, typically via a mapping table that becomes part of the cross-zone integration relation. This preserves the human-in-the-loop guarantee that enterprise integration demands.

Cross-zone integration relations

Two sibling zones compose via a **cross-zone integration relation** — a 4-tuple $(\gamma, C, \delta, \epsilon)$:

- γ — a set of inter-zone role mappings that transfer permissions between subsystems.
- C — a set of shared constraints (access, identity, trigger, goal) that both subsystems must respect.
- δ — a set of cross-zone daemons that monitor the interaction between the two subsystems.
- ϵ — a cross-zone embedding alignment that projects both subsystems' vocabularies into a shared space.

```
# Add an integration relation between two sibling subsystems
fed.integrate(
    from_zone=legacy_hr_a,
    to_zone=legacy_hr_b,
    gamma=[
        (legacy_hr_a.roles["HRViewer"],
         legacy_hr_b.roles["HRViewer"],
         1.0),
    ],
    shared_constraints=[
        """forall u: User .
           not (hasRole(u, HRViewer) and hasRole(u, FinanceApprover))""",
    ],
    cross_zone_daemons=[CrossRegionDriftDaemon()],
    embedding_alignment=True,
)
```

The Integration Theorem

Federation is formally sound. The Integration Theorem (Theorem 5 in the paper) guarantees that:

Let S_1 and S_2 be two independent, soundly integrated CZOI systems with no shared operations. Then the composed federation $S = S_1 \oplus S_2$ under a new common parent satisfies:

1. S is a well-formed CZOI system.
2. Every identity constraint of S_1 and S_2 holds in S .
3. Every access constraint of S_1 and S_2 holds in S .
4. Any additional constraint on the parent is enforced for both subsystems.

The corollary applies directly to mergers:

Corollary (Post-Merger Integration Guarantee). In a post-merger scenario where two legacy organizations are integrated under a new parent authority, if both legacy systems are soundly integrated CZOI systems, the merger preserves each legacy system's security guarantees while enabling explicit cross-system policy at the authority level.

Three integration patterns

Pattern	Categorical name	Use case
Parallel integration	Product ($\times$)	Post-merger integration, multi-tenant SaaS, federated identity
Alternative integration	Coproduct ($+$)	A/B testing, blue-green deployment, geographic failover
Governed integration	Exponential ($\rightarrow$)	Parent governs subsidiary, regulator supervises entity

End-to-end integration example

Let's federate a legacy HR system and a new finance system into a single federated organization.

```
from czoi import (
    AdapterZone, CZOABuilder, FederationBuilder, Operation, Role, User,
)

# ---- 1. Wrap the legacy HR system ----
legacy_hr = AdapterZone(name="LegacyHR", adapter=MyHRAdapter())
legacy_hr.expose_operations({
    "view_employee": Operation("view_employee"),
})
legacy_hr.add_role(Role("HRViewer", zone=legacy_hr,
    base_permissions=
[list(legacy_hr.operations.values())[0]))

# ---- 2. Build the new finance system ----
finance_builder = CZOABuilder("Finance")
finance_app = Application("FinanceApp", zone=finance_builder.root)
approve = finance_app.add_operation(Operation("approve_invoice"))
finance_builder.root.add_application(finance_app)
finance_builder.root.add_role(
    Role("FinanceApprover", zone=finance_builder.root,
        base_permissions=[approve])
)

# ---- 3. Federate under a shared parent ----
fed = FederationBuilder("MergedCorp")
fed.adopt_subsystem(legacy_hr, name="LegacyHR")
fed.adopt_subsystem(finance_builder, name="Finance")
```

```
# ---- 4. Share a separation-of-duty constraint -----
fed.share_constraint("""
    signature {
        sort User, Role;
        constant HRViewer : Role;
        constant FinanceApprover : Role;
        predicate hasRole(u: User, r: Role);
    }
    forall u: User .
        not (hasRole(u, HRViewer) and hasRole(u, FinanceApprover))
""")

# ---- 5. Create users in the federation -----
alice = User("alice", roles={"HRViewer"})
fed.root.add_user(alice)
legacy_hr.add_user(alice)

bob = User("bob", roles={"FinanceApprover"})
fed.root.add_user(bob)
finance_builder.root.add_user(bob)

# ---- 6. The federation is a CZOI system -----
engine = fed.permission_engine
print(engine.decide(alice, list(legacy_hr.operations.values())[0],
                    legacy_hr).name)    # ALLOW
print(engine.decide(bob, approve, finance_builder.root).name)    # ALLOW

# A user with both roles would be denied by the shared SoD constraint.
charlie = User("charlie", roles={"HRViewer", "FinanceApprover"})
fed.root.add_user(charlie)
legacy_hr.add_user(charlie)
finance_builder.root.add_user(charlie)
print(engine.decide(charlie, approve, finance_builder.root).name)    # DENY
```

Cross-zone monitoring

```
from czoim import Daemon, DaemonSignal

class VocabularyDriftDaemon(Daemon):
    """Detects when two subsystems' role vocabularies diverge."""

    def __init__(self, zone_a, zone_b, threshold=0.2, parent=None):
        super().__init__("VocabularyDrift", parent=parent, interval=60.0)
        self.zone_a = zone_a
        self.zone_b = zone_b
        self.threshold = threshold

    def monitor(self):
        roles_a = set(self.zone_a.roles.keys())
        roles_b = set(self.zone_b.roles.keys())
        drift = len(roles_a.symmetric_difference(roles_b)) \
```

```

        / max(len(roles_a | roles_b), 1)
    if drift > self.threshold:
        self.emit_signal(
            DaemonSignal.STATE_WARNING,
            {"event": "vocabulary_drift", "drift": drift},
        )

```

Verifying integration soundness

```

from czoifederation import verify_integration

report = verify_integration(fed)
print(report.summary())
# Integration Report
# -----
# Well-formedness      : OK
# Identity preservation : OK
# Access preservation  : OK
# Constraint flow      : OK
# Cross-zone daemons   : OK
# Audit completeness   : OK

```

Real-world impact

In the paper's post-merger case study, a CZOI federation formed from two legacy health authorities achieved a **61 % reduction in integration time**, **100 % elimination of duplicate role definitions**, and **zero modifications to the legacy systems**. Both legacy systems retained their compliance certifications, and cross-region specialist consults — impossible in the pre-merger state — became available immediately. Permission latency across regions dropped from 4.2 ms to 0.42 ms.

Exercises

1. **Wrap a legacy system.** Take any existing system you know (or invent one). Define three operations, two roles, and one property it would expose as an adapter zone. Write the adapter class skeleton (interface only, no implementation).
2. **Federate two systems.** Build two small native CZOI systems (2 zones, 2 roles, 2 operations each). Federate them under a new parent. Verify that a user in one subsystem can access an operation in the other only if the corresponding cross-zone integration relation is defined.
3. **Discover equivalence.** Build two systems with semantically equivalent roles under different names (e.g., **Doctor** and **Physician**). Use embeddings to discover the equivalence.
4. **Shared SoD constraint.** Add a shared separation-of-duty constraint to the federation that blocks any user from holding both roles from Exercise 2. Verify the constraint fires.
5. **Integration Theorem by example.** Take a small federation and write a test that verifies properties (1)–(4) of the Integration Theorem for your specific system.

Further reading

- The CZOA paper, §3.4 (Integration Semantics)

- The CZOA paper, Theorem 5 and Corollary 1 (Integration Theorem)
- The CZOA paper, §7.7 (Post-merger case study)
- The User Guide, §14 (Integrating Heterogeneous Systems)

Part VI — Advanced

Chapter 16 — Cross-Zone Reasoning

Learning objectives

- Implement inter-zone role mappings (γ).
- Use embeddings for cross-zone role matching.
- Handle cross-zone operations.

Sometimes a user in one zone needs a permission defined in another. The paper models this with γ (**gamma**) **mappings** — inter-zone role relations. The cleanest implementation is an explicit grant:

```
def add_gamma_mapping(
    child_zone, child_role,
    parent_zone, parent_role,
    weight=1.0,
):
    """Grant child_role the operations of parent_role."""
    parent_ops = sorted(parent_role.base_permissions,
                        key=lambda o: o.qualified_name)
    n_transfer = max(1, int(round(weight * len(parent_ops))))
    for op in parent_ops[:n_transfer]:
        child_role.grant(op)
    child_zone._invalidate_cache()

# Usage: Developer inherits Engineer's permissions during on-call
add_gamma_mapping(dev_zone, developer, corp_zone, engineer, weight=1.0)
```

Every γ mapping should be reversible — cross-zone access granted during a crisis should be revoked when the crisis passes.

γ mappings are the integration primitive

In a federation, γ mappings are the mechanism by which permissions flow across subsystem boundaries. They are also the mechanism by which the federation controls which cross-system operations are permitted — no γ , no cross-zone access. The discovered role equivalences from Chapter 15 become γ mappings, reviewed and applied by a human administrator.

Exercises

- γ mappings transfer permissions between roles in different zones.
- Weight controls the fraction of permissions transferred; deterministic for reproducibility.
- Every γ mapping should be reversible.

- **Exercise:** Build an on-call rotation where a single role inherits from multiple service roles at different weights.

Further reading

- The CZOA paper, §3, item 2 (γ mappings)
- The User Guide, §14 (Integrating Heterogeneous Systems)

Chapter 17 — Composition and Federation

Learning objectives

- Compose two CZOI systems into a larger system.
- Understand the categorical structure (products, coproducts, exponentials).
- Reason about emergent behaviour.

CZOA is **compositional**: any two CZOI systems can be combined into a larger CZOI system. This is what makes it a **system-of-systems** architecture — and it is the same property that makes federation sound.

Composition operators

- **Product ($\times$)** — parallel composition. Both subsystems exist, independently.
- **Coproduct ($+$)** — alternative composition. One or the other.
- **Exponential ($\rightarrow$)** — morphism objects. One subsystem governs another.

These are the three integration patterns from Chapter 15, expressed in categorical terms.

Clone and product

```
from czoitutorial import CompositeZone, Role, Application, Operation

original = CompositeZone("Original")
app = Application("App", zone=original)
op = app.add_operation(Operation("do"))
original.add_application(app)
original.add_role(Role("Doer", zone=original, base_permissions=[op]))

clone = original.clone(name="Clone")
print(clone.roles["Doer"] is original.roles["Doer"])      # False — new
Role
print(clone.operations["App.do"] is op)                    # True — shared
Operation
```

Why share operations but not roles? Because operations are immutable (their identity is their qualified name) while roles are mutable (they carry permissions). Cloning roles isolates mutations.

Emergent properties

Composition can produce **emergent properties** — things that are true of the whole but not of any part. Two zones each running at 40 % capacity might be fine individually, but together consume 80 % of shared resources. Detect this with a cross-zone daemon:

```
class ResourceCoordinator(Daemon):
    def monitor(self):
        total_util = sum(
            z.properties.get("utilisation", 0) for z in self.zones
        )
        if total_util > self.threshold:
            self.emit_signal(
                DaemonSignal.STATE_CRITICAL,
                {"total_utilisation": total_util},
            )
```

Federation is the product pattern

Federation — the integration pattern from Chapter 15 — is exactly the categorical **product** operation. Two subsystems are composed under a new parent without modifying either. The categorical laws (products commute, are associative, have an identity) apply directly to integration: you can federate subsystems in any order, at any time, and the composed system remains well-formed.

Exercises

- CZOA systems form a Cartesian closed category.
- Products are parallel composition (federation); coproducts are alternative; exponentials are governed.
- Products commute and are associative — composition order doesn't matter.
- **Exercise:** Build two small systems; merge them under a holding company; verify constraints from the holding apply to both.

Further reading

- The CZOA paper, §4 (Category-theoretic foundations)
- The User Guide, §5 (Building Zone Hierarchies)

Chapter 18 — The Category-Theoretic View

Learning objectives

- Understand the categorical foundations of CZOA.
- Appreciate why the formalism is more than bookkeeping.
- See how the categorical view enables compositional reasoning.

Category theory is the mathematics of structure. CZOA uses it to make precise claims: "Any two CZOI systems can be composed"; "Composition preserves constraints"; "Integration is a morphism." These have formal proofs and algorithmic consequences.

The category CZOA_Rec

- **Objects:** recursive CZOI systems.
- **Morphisms:** structure-preserving maps between systems.
- **Products, coproducts, exponentials:** the categorical primitives that correspond to integration patterns.

The Integration Functor

The paper introduces $F_integr: CZOA_Rec \rightarrow Graph$, mapping each system to its **integration graph** whose vertices are zones and whose edges are cross-zone integration relations. Because F_integr is a functor, any morphism between systems induces a graph homomorphism between their integration graphs — integration relations are preserved under system transformations.

This is what makes CZOI federation *compositional* rather than merely *additive*. You can refactor one subsystem and be confident that its integration relations with the others remain coherent.

Why integration is provably sound

The Integration Theorem's guarantees are not accidents. They follow from the recursive structure of CZOI: containment ensures authority flows downward; local decision functions ensure autonomy flows upward; operation-level interfaces ensure composition is well-defined; non-interference preserves each subsystem's invariants. These four properties are what distinguish CZOA federation from ad-hoc integration — in an ad-hoc federation, you can only *assert* that integration works; in CZOA, you can *prove* it.

Exercises

- The functor F_integr maps each system to its integration graph.
- Functors compose — a transform of one subsystem induces a transform of its integration relations.
- You don't need category theory in daily work, but its guarantees are why the framework behaves predictably.
- **Exercise:** For the federation in Chapter 15, draw the integration graph (zones as nodes, integration relations as edges).

Further reading

- The CZOA paper, §4 (Category-theoretic foundations)
- Fong & Spivak, *An Invitation to Applied Category Theory: Seven Sketches in Compositionality* (free online)

Conclusion

You've reached the end of the CZOA tutorial. Let's take stock of what you've learned.

The journey

You started with a problem: three fragmentations — intelligence vs. security, autonomy vs. governance, and heterogeneous systems vs. coherent operations — that cost real organizations real resources. You learned that CZOA closes all three fragmentations with a single recursive mechanism.

You built up the theory in layers:

1. **Zones** — recursive systems that mirror organisational structure, and that can also represent heterogeneous subsystems through adapter zones.
2. **Roles, users, operations** — the identity and capability model.
3. **The permission calculus** — two-stage recursive decisions that compose across subsystem boundaries.
4. **Neural components** — learnable functions inside every zone; local models that compose without global retraining.
5. **Embeddings** — semantic vectors for cross-zone reasoning; the mechanism that bridges heterogeneous vocabularies.
6. **Adaptive access control** — dynamic permissions with safety guarantees.
7. **Constraints** — formal policy in UniLang; hierarchical, so shared governance is imposed automatically.
8. **Daemons** — continuous monitoring and coordinated response, including cross-zone monitoring for integration health.
9. **Federation** — the formal composition of heterogeneous subsystems under shared governance, with the Integration Theorem guaranteeing preservation of each subsystem's security invariants.

Then you learned to apply it:

- **Designing for a domain** — a step-by-step process.
- **Testing** — how to verify CZOI systems.
- **Deployment** — four patterns from embedded to hybrid.
- **Integrating heterogeneous systems** — adapter zones, federation, and the Integration Theorem.
- **Cross-zone reasoning** — γ mappings and semantic matching.
- **Composition** — systems of systems and categorical laws.

What you can do now

- Look at any organisation and design a CZOI model.
- Write formal constraints that capture policy — and compose them across subsystems.
- Build adaptive systems that respond to changing conditions.
- Wrap legacy systems as adapter zones without modifying them.
- Federate independent subsystems under shared governance with formal guarantees.
- Verify your designs with tests and analysis.
- Deploy CZOI systems in production — as federations when necessary.

The bigger picture

The organizations that thrive in the next decade will not be those that chose between autonomy and integration, nor between security and intelligence. They will be those that discovered how to have all four. The CZOI toolkit is one path to that discovery.

What to explore next

- **Federated learning across CZOA subsystems** — the natural extension of local neural components to cross-organizational training.
- **Conflict-resolution algebra for federations** — what to do when two subsystems have contradictory constraints.
- **Semantic drift detection** — automated systems for detecting when two subsystems' vocabularies have diverged.
- **Automated zone discovery** — inferring zone trees from organizational data.
- **Daemon synthesis** — generating monitoring daemons from high-level policies.

Getting help

- **The User Guide** — reference details, including the integration guide.
- **The repository** — source code and examples.
- **The paper** — formal foundations.
- **GitHub issues** — questions and bug reports.

Contributing

If you build something interesting with CZOI — a new pattern, a new integration scenario, a new domain — consider contributing it back. The toolkit grows with its users.

Appendices

Appendix A — Quick Reference

Building

```
from czi import (
    CZOABuilder, Application, Operation, Role, User,
    AdapterZone, FederationBuilder,
    Daemon, DaemonSignal, Predictor, AnomalyDetector, RoleMiner,
    EmbeddingService, Decision, NeuralContribution,
)

builder = CZOABuilder("Name")
zone = builder.add_zone("ZoneName", parent=builder.root, atomic=False)
```

Integrating a legacy system

```
adapter_zone = AdapterZone(name="LegacySystem", adapter=MyAdapter())
adapter_zone.expose_operations({"op_name": Operation("op_name")})
adapter_zone.add_role(Role("RoleName", zone=adapter_zone,
                           base_permissions=[...]))
```

Federating

```

fed = FederationBuilder("MergedCorp")
fed.adopt_subsystem(zone_a, name="SubsystemA")
fed.adopt_subsystem(zone_b, name="SubsystemB")
fed.share_constraint("""...""")
fed.add_cross_zone_daemon(MyFederationDaemon())

```

Roles, operations, users

```

app = Application("App", zone=zone)
op = app.add_operation(Operation("do_thing"))
zone.add_application(app)

role = Role("Role", zone=zone, base_permissions=[op])
zone.add_role(role)
role.add_junior(junior_role)

user = User("user", roles={"Role"}, attributes={"key": "value"})
for z in zone.ancestry():
    z.add_user(user)

```

Permissions, constraints, daemons, neural

```

decision = builder.permission_engine.decide(user, op, zone)

builder.add_access_constraint("""... UniLang ...""")

class MyDaemon(Daemon):
    def monitor(self):
        self.emit_signal(DaemonSignal.STATE_WARNING, {"key": "value"})
    def on_signal(self, signal, payload, source=None):
        pass
builder.add_daemon(MyDaemon("Name"))

model = Predictor("name", threshold=0.5)
model.fit(X, y, epochs=500)
zone.add_neural("name", model)

```

Integration utilities

```

from czi.federation import verify_integration, discover_role_equivalences

report = verify_integration(fed)
print(report.summary())

equivalences = discover_role_equivalences(

```

```
zone_a, zone_b, embedding_service=emb, threshold=0.75,
)
```

Appendix B — Glossary

- **Adapter Zone** — a CZOI zone that wraps a non-CZOI subsystem through a declared interface.
- **AtomicZone** — a leaf zone with no children.
- **Application** — a structural module grouping operations.
- **CompositeZone** — a zone that may have children.
- **Containment principle** — $U_{child} \subseteq U_{parent}$.
- **Decision** — **ALLOW**, **DENY**, or **INCONCLUSIVE**.
- **Effective permissions** — the union of base and inherited permissions.
- **Federation** — a CZOI system with multiple operationally independent child zones.
- **γ (gamma) mapping** — an inter-zone role permission transfer.
- **Integration relation ($\mathfrak{I}$)** — a 4-tuple $(\gamma, C, \delta, \epsilon)$ defining how two sibling zones compose.
- **Neural component** — a learnable function inside a zone.
- **Operation** — an atomic permission target.
- **Role** — a job function within a zone.
- **Seniority** — the intra-zone role hierarchy.
- **UniLang** — the formal constraint language.
- **Zone** — a recursive CZOI subsystem.

Appendix C — Further Resources

Papers

- Wang, H. (2025). Constrained Object Hierarchies as a Unified Theoretical Model for Intelligence and Intelligent Systems. *Computers*, 14(11), 478.
- Wang, H. (2026). A Formalized Zoned Role-Based Framework. *Computers*, 15(3), 1–26.
- Wang, H. (2026). Constrained Zoned-Object Architecture (CZOA): A Unified Framework for Integrating Fragmented Systems, Secure Access, and Organizational Intelligence. *Preprint*.
- Wang, H. (2026). UniLog: A Unified Logic Framework. *Preprint*.

Code

- [CZOI Toolkit](#)
- [UniLog Toolkit](#)

Background

- Beer, S. (1972). *Brain of the Firm*. Allen Lane. — The Viable System Model.
- Fong, B. & Spivak, D. (2019). *An Invitation to Applied Category Theory*. Cambridge.
- Greefhorst, D. & Proper, E. (2011). *Architecture Principles*. Springer.
- Ross, J. W., Weill, P., & Robertson, D. C. (2006). *Enterprise Architecture as Strategy*. Harvard Business Review Press.
- Lankhorst, M. (2017). *Enterprise Architecture at Work* (4th ed.). Springer.

End of tutorial. Thank you for reading.